1 Access, Users and Passwords

For security and centralized configuration, credentials are injected with Docker secrets mounted at `/run/secrets`. For testing purposes within this laboratory setup, default passwords are standardized to `groupH`, the only exception is the Moodle administrator, because Moodle refuses passwords that do not follow its policy (upper and lower case, digit, and symbol).

1.1 Web Addresses

Service	Local (`compose.yml`)	Production (`production.yml`)	Auth
Reviews application	`http://app.localhost:8080`	`https://app.localhost`	Public
Moodle (Atenea)	`http://lms.localhost:8080`	`https://lms.localhost`	Required
Grafana	`http://grafana.localhost:8080`	`https://grafana.localhost`	Required
Mailpit	`http://mail.localhost:8080`	`http://mail.localhost:8080`	Public

Table 1: Deployed web endpoints and authentication requirements.

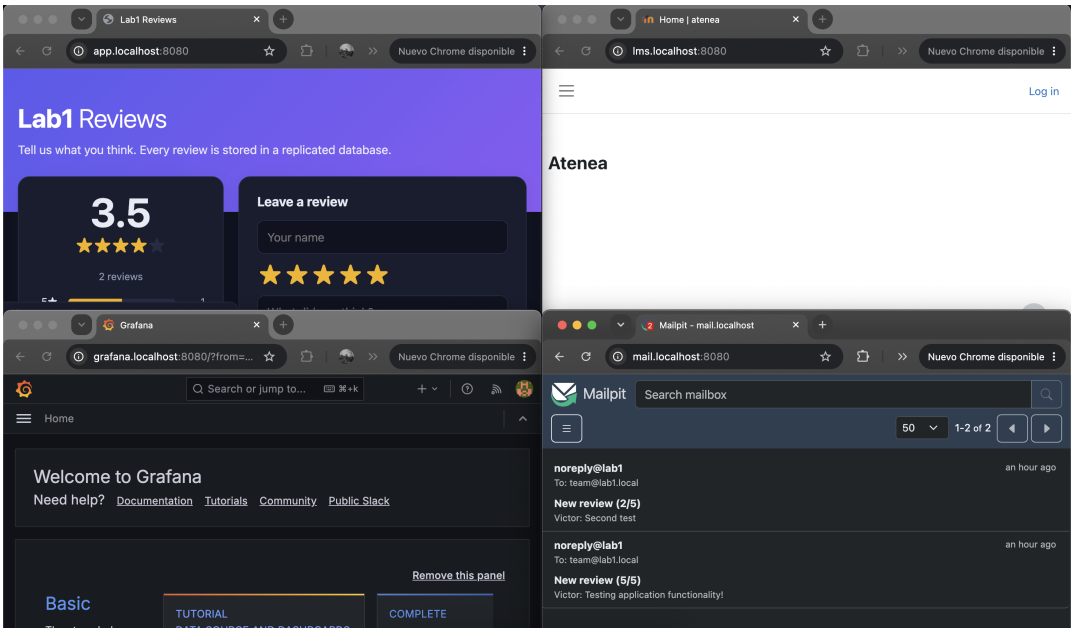


Figure 1: Running deployment of the web addresses.

1.2 Users and Passwords

Table 2 shows the service credentials, secret files, and the configured database access scopes.

Service	User	Password	Secret File
Grafana	admin	groupH	grafana_admin_password
Moodle	admin	groupH-2026	moodle_admin_password
PostgreSQL (Patroni)	postgres	groupH	pg_admin_password (Internal HA management)
PostgreSQL (App)	app	groupH	app_db_password (Owner of app DB)
PostgreSQL (Grafana)	grafana	groupH	grafana_db_password (Owner of grafana DB)
PostgreSQL (Replication)	replicator	groupH	replicator_password
PostgreSQL (Backup)	backup	groupH	backup_db_password (pg_read_all_data)
PostgreSQL (Metrics)	exporter	groupH	exporter_db_password (pg_monitor)
MariaDB (Moodle)	moodle	groupH	moodle_db_password
MariaDB (Auxiliary)	backup, exporter	groupH	Shared secrets (Read-only access)
Backup Encryption	—	groupH	backup_passphrase (AES-256-CBC)
Self-signed TLS key	—	groupH	CERT_PASSWORD
Feeder API	—	groupH	feed_token (Bearer Token)

Table 2: System account roles and Docker secret mappings.

2 Data Management and Backups

To avoid database corruption caused by network disruptions or file-locking conflicts, transactional DBMS files are stored directly on local node drives rather than over NFS.

Data Target	Volume Name	Physical Node Location
PostgreSQL Cluster & etcd	pg-1..3, etcd-1..3	Local disk (app1, app2, stats)
MariaDB & NFS Root	moodle-db, nfs-data	Local disk (atenea)
Moodle Data & Certificates	moodledata, certs, acme	NFS Storage (atenea)
Encrypted Backups	backups	Dedicated disk (backup)
Prometheus Replicas	Anonymous volumes	Local disk (stats, backup)
Redis Cache	In-Memory	Replicated RAM

Table 3: Physical storage distribution across Swarm nodes.

The backup service operates on a dedicated host isolated from core application nodes. Scheduled daily at 03:30, the pipeline performs:

1. **PostgreSQL Dump**: Uses `pg_dump -Fc` routed to read-only replicas (`db:5433`) under the `backup` role to prevent load on the HA leader.
2. **MariaDB Dump**: Executes `mariadb-dump -single-transaction` for transactional consistency.
3. **Moodle Data Storage**: Archives `moodledata` while stripping volatile session and cache directories.
4. **Encryption & Integrity**: Encrypts files using OpenSSL AES-256-CBC (PBKDF2) with the secret passphrase and generates matching `SHA256SUMS`.

Backups are initially written with a `.partial` suffix and atomic-renamed upon completion, and retention is configured for 14 days.

An automated integrity check runs every Sunday (and on demand with `docker exec <backup> /restore-test.sh`). The routine verifies checksums, decrypts archives, restores dumps into temporary containerized DBMS instances, and validates recovered row and table counts. Test outcomes are exported to Prometheus (`backup_restore_test_ok`).

3 Observability

Prometheus uses DNS service discovery (`tasks.<service>`) so scaled replicas are scraped without manual changes. Dashboards are provisioned declaratively via Git, where Grafana visualizes node metrics alongside custom ratings using the Infinity plugin. Additionally, Prometheus alerting rules (`rules.yml`) flag offline endpoints, replication delays, missing cluster leaders, and backup failures. Table 4 summarizes the metric sources and monitored targets.

Metric Source	Monitored Target	Key Operational Indicators
`core`	Python Microservice	Requests/sec, posted reviews, Redis cache hit/miss ratio
Nginx Exporter	Frontend Proxy	Active connections, throughput per replica
Postgres Exporter / Patroni	PostgreSQL Nodes	Connection pool, transactions, replication lag, leader status
Redis Exporter	Cache Instances	Master/Replica role, memory consumption, hit ratio
MySQLd Exporter	MariaDB Instance	Active threads, queries per second (QPS)
Blackbox Exporter	Web Endpoints	Availability and HTTP response latency for Moodle/Grafana/App
cAdvisor & Node Exporter	Host Nodes	Node-level CPU, RAM usage, and container resource limits
Backup Exporter (9199)	Backup Service	Last execution age, duration, archive size, restore status

Table 4: Observability exporters and monitored metrics.

Figure 2 presents the main Grafana overview dashboard after simulating application traffic generated by user interactions with the reviews app.

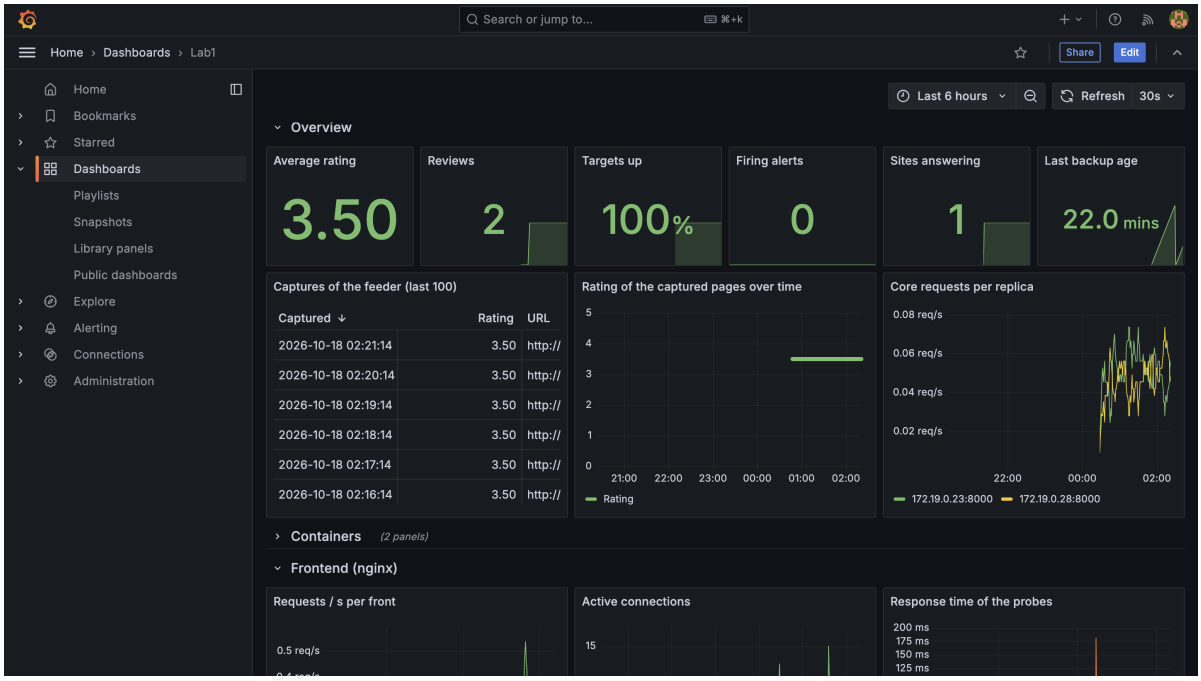


Figure 2: Grafana monitoring dashboard.

4 Security

- **Traffic Control**: Only Nginx exposes external ports (80/443). All traffic is forced through TLS 1.2/1.3 with HSTS enabled. Internal microservices remain isolated.

- **Least-Privilege Database Access**: Database owners (`app`, `grafana`) are restricted to their respective databases. The `replicator` user is confined to replication streams, and global `CONNECT` rights are revoked from `PUBLIC`.
- **Encrypted Inter-Node Communication**: Secrets are mounted in-memory at `/run/secrets`. Overlay networks utilize IPsec encryption, segregating individual application stacks.
- **Metrics Isolation**: Metric endpoints (`/metrics`) are blocked at the proxy layer, preventing external exposure.